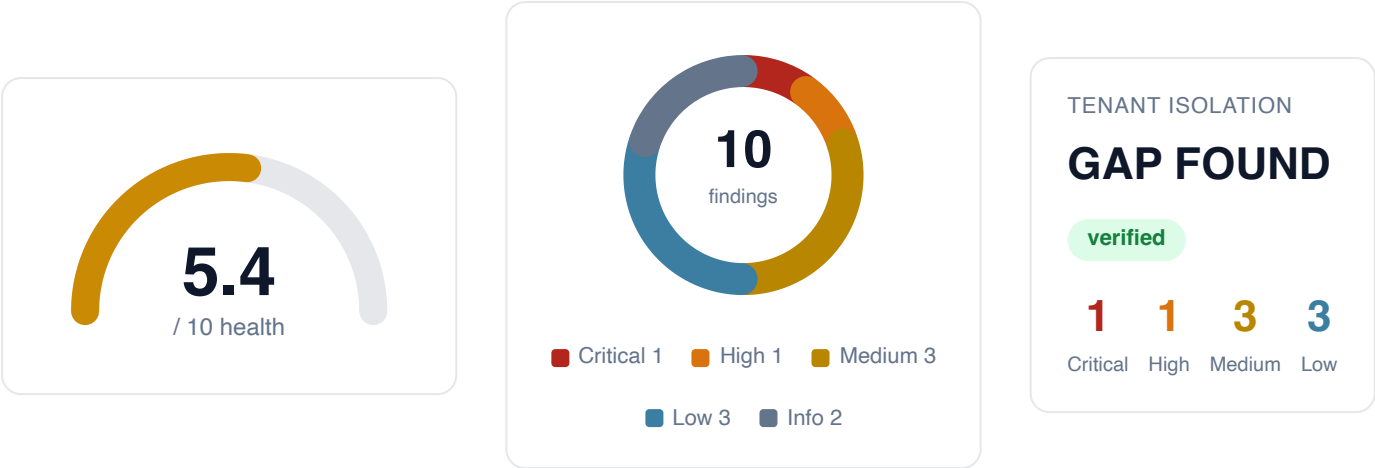


Security & Health Audit

Larkspur (sample) — Multi-tenant Supabase / Next.js booking app
2026-07-30 · sample @ demo · Prepared by Harvey (harvey-qa.com)



Partial audit — 9/10 modules fully assessed. Not fully run: M10 (partial). See Module coverage below for reasons.

One cross-tenant read was proven live and one N+1 will not survive traffic. The rest are hardening, test-quality, and maintainability wins — all with named fixes.

Top bang-for-the-buck

- 80

Cross-tenant read proven: user A can read user B's bookings
- 80

N+1 query on the dashboard loads one query per booking
- 80

The refund guard has a test that can't fail
- 38

service_role key reachable from a client-imported module
- 32

Date-formatting logic copy-pasted across six components
- 29

PII sitting in three columns with no access policy

Action plan

Everything BFTB > 75, plus every Critical/High security finding.

#	ACTION	WHY	BFTB	OWNER
1	Rewrite the policy to scope by owner: USING (user_id = auth.uid()). Re-run the cross-tenant matrix to confirm the boundary now holds.	Cross-tenant read proven: user A can read user B's bookings — Critical	80	Operator
2	Replace the per-row lookup with a single joined query (select with an embedded resource), or batch the customer IDs into one in() query.	N+1 query on the dashboard loads one query per booking — High	80	Eng

3	Add one test asserting a refund greater than the original charge is rejected.	The refund guard has a test that can't fail — Medium	80	Eng
---	--	--	----	-----

Scope & methodology

Reviewed RLS policies, service-role code paths, API routes, auth/permission layer, the booking and billing modules, and the test suite.

Tooling A live two-tenant stand-up (M2), Supabase RLS/advisor review (M1), vitals (hotspots), knip (dead code), jscpd (duplication), Stryker (mutation testing on money-critical code), and the M9 static Server Action/boundary detectors. Every finding passes a correctness check (repro/evidence) and an applicability check.

Module coverage

All ten audit modules, and whether each was assessed on this engagement. Every module was assessed.

MODULE	AREA	STATUS	WHAT RAN / WHY NOT
M1	Multi-tenant security	Assessed	RLS + auth reviewed, live-verified
M2	Local pen-test (dynamic)	Assessed	stack stood up, cross-tenant matrix run
M3	Hotspot analysis	Assessed	churn × complexity computed
M4	Duplication	Assessed	jscpd + near-miss pass
M5	Slop / dead code	Assessed	knip over installed deps
M6	Simplification / maintainability	Assessed	indicators + reviewed verdict
M7	Performance	Assessed	code tier + DB advisors + Lighthouse
M8	Test quality	Assessed	mutation run complete
M9	App Router boundary/rendering	Assessed	static AST pass
M10	Data classification (PII/PHI/PCI)	Partial	schema classification ran; live PII-grant check pending client DB sign-off

Out of scope Infrastructure, third-party dependencies, social engineering.

Confidence & limitations

What each module's result does and does not prove — independent of whether it ran this engagement (see Module coverage above for that).

MODULE	AREA	KNOWN LIMITATIONS
M1	Multi-tenant security	Mechanical/config detectors are pattern-based against a calibration corpus, not exhaustive. LLM semantic findings carry a Confidence tag (Likely/Review) until manually confirmed. Live RLS/policy review needs connected-tier DB access.
M2	Local pen-test (dynamic)	Proves isolation against a freshly seeded local stack with synthetic tenants — confirms the app as configured, not production data or config drift since deploy.
M3	Hotspot analysis	Churn × complexity ranking depends on git history depth; a shallow clone or young repo yields a thin signal.
M4	Duplication	Mechanical clone detection; some duplication may be intentional (generated code, fixtures) — reviewed, not auto-excluded.

M5	Slop / dead code	Flags candidate unused exports/code mechanically; some may be public API or dynamically referenced — review before pruning.
M6	Simplification / maintainability	The free tier is non-grading indicators only. The paid verdict is a reviewed judgment over a packet — an unreviewed packet is not a verdict.
M7	Performance	Code-tier findings are AST pattern matches and a share of them do not survive review. Read against source on six pinned open-source codebases: 47 of 65 graded findings held (2026-07-28, 72%); after six named false-positive families were fixed, 48 of the 50 findings that still emit hold — but on a residual of only 50 rows, and one class (nested-loop join) has a recorded open disagreement about its true rate. A 40-row random sample drawn from 858 graded findings on four larger codebases held 78% before those fixes and has not been re-sampled since. Treat the code-tier list as a strong lead list that still needs reading, not as a verified defect list. Per-class precision is NOT stated for classes with too few field instances to measure — an unquoted class is unmeasured, not clean. Info-severity notes (hook dependencies, and since 2026-07-28 the state-sprawl maintainability observation) are excluded from these counts and are not graded. DB advisor findings reflect the platform's own heuristics, not independently re-verified. Core Web Vitals/Lighthouse figures are a single LAB run, not field/real-user data.
M8	Test quality	A mutation score is only as good as its stated scope — a scoped run is not a whole-repo coverage claim (see Test quality below when present).
M9	App Router boundary/rendering	AST pattern coverage of App Router conventions, not full framework-semantics execution.
M10	Data classification (PII/PHI/PCI)	Classification is derived from column/schema naming and structure. A JSON/JSONB container is flagged for review, not asserted, and its actual contents are not read.

Test quality & intent (M8)

Mutation testing (StrykerJS): **your coverage number is lying to you — here's where**. Line coverage says a line RAN; the mutation score says a fault injected into it would FAIL a test. A module with high coverage and a low mutation score has tests that execute the code without asserting on it.

71 % mutation

84 % of COVERED code

Covered scope `lib/billing/**` , `lib/bookings/**`

This score is measured over the file(s) above only — a scoped subset, NOT a whole-repo coverage claim. Untested files do not appear in this number.

Scoped to money-critical and booking-core modules for this sample.

MODULE / FILE		LINE COV	MUTATION SCORE	SURVIVING MUTANTS (CRITICAL)	ACTION
lib/billing/refund.ts	False confidence	92%	71%	0 of 1	Add a test asserting a refund greater than the original charge is rejected.

Surviving mutants — the tests that can't fail. Each row is a change we made to your code that your suite did not notice. Hotspot-flagged ones sit in churny, complex, security-relevant code (cross-referenced with M1/M3), so they lead.

LOCATION	MUTATOR (WHAT CHANGED)	ON HOTSPOT
lib/billing/refund.ts:42	ConditionalExpression	—

Findings

F-01 Cross-tenant read proven: user A can read user B's bookings

Critical

BFTB 80

Confirmed

Security · cross-tenant-read · `public.bookings (RLS policy "read own bookings")` · Open · V5-E4-S5

Evidence On the stood-up stack, signed in as tenant A, a request to `/rest/v1/bookings` returned tenant B's rows. The policy uses `USING (auth.role() = 'authenticated')` — it checks that you're logged in, not that the row is yours.

Impact Any signed-in customer can read every other customer's bookings, including names, dates, and contact details. This is a direct data-exposure incident, not a theoretical one.

Fix Rewrite the policy to scope by owner: `USING (user_id = auth.uid())`. Re-run the cross-tenant matrix to confirm the boundary now holds.

F-02 N+1 query on the dashboard loads one query per booking

High

BFTB 80

Confirmed

Performance · n-plus-one-query · `app/dashboard/page.tsx` · Open · V5-E4-S5

Evidence The dashboard fetches the booking list, then issues a separate customer lookup inside the render loop — N+1 queries that scale linearly with row count.

Impact At 20 bookings the page is fine; at 2,000 it issues 2,000 queries per load and times out. This is what falls over on launch day.

Fix Replace the per-row lookup with a single joined query (select with an embedded resource), or batch the customer IDs into one `in()` query.

F-04 The refund guard has a test that can't fail

Medium

BFTB 80

Confirmed

Test quality · tests-verify-intent · `lib/billing/refund.ts` · Open · V4-E5-S5

Evidence Mutation testing: deleting the `amount <= originalCharge` check left every test green. No test asserts that an over-refund is rejected on money-critical logic.

Impact A regression that allows refunding more than was charged would ship undetected — the suite would still pass.

Fix Add one test asserting a refund greater than the original charge is rejected.

F-03 service_role key reachable from a client-imported module

Medium

BFTB 38

Likely

Security · service-role-exposure · `lib/supabase/admin.ts` · Open · V4-E3-S4

Evidence The admin client is instantiated with the `service_role` key in a module that is transitively imported by a client component. It is not currently called from the browser, but nothing prevents it.

Impact The `service_role` key bypasses RLS entirely. A future accidental client import would bundle a master key to the database into the browser.

Fix Move the admin client behind a server-only boundary and add import `'server-only'` so an accidental client import fails the build.

F-05 PII sitting in three columns with no access policy

Medium

BFTB 29

Confirmed

Data classification · pii-broad-grant · `customers.phone, customers.address, customers.date_of_birth` · Open · V3-E3-S4

Evidence Schema classification flagged three PII columns on a table whose RLS is enabled but whose `SELECT` grant is broader than the policy intends.

Impact Personal contact data is more broadly readable than the app appears to intend — a privacy and compliance exposure.

Fix Tighten the `SELECT` grant to match the intended policy, and confirm no client path reads these columns unnecessarily.

F-07 Date-formatting logic copy-pasted across six components

Low

BFTB 32

Confirmed

Duplication · clone-detection · `components/**` (6 clones) · **Open** · V2-E4-S5

Evidence jscpd: the same timezone-handling block appears in six components, already diverged in two of them.

Impact A fix to one copy silently misses the others — the two that diverged already render times differently.

Fix Extract one `formatBookingTime` helper and replace the six copies.

F-06 A Server Action mutates without checking the caller

Low

BFTB 29

Confirmed

Security · server-action-idor · `app/bookings/actions.ts` (`cancelBooking`) · **Open** · V3-E3-S4

Evidence The Server Action accepts a `bookingId` and cancels it with no ownership check — it trusts that the UI only offers the button on the user's own bookings.

Impact Anyone who can call the action with an arbitrary `bookingId` can cancel another tenant's booking (IDOR via a Server Action).

Fix Verify the booking belongs to `auth.uid()` inside the action before mutating; never rely on the UI to enforce authorization.

F-08 Hand-rolled session parsing where a library exists

Low

BFTB 19

Confirmed

Maintainability · reinvented-library · `lib/auth/session.ts` · **Open** · V2-E3-S4

Evidence Custom cookie parsing and JWT decoding reimplement what the Supabase auth-helpers already provide, with an edge case that mishandles an expired token.

Impact Fragile reinvented auth code — the exact category that quietly breaks and is hard to reason about.

Fix Replace with the maintained Supabase auth-helpers session handling.

F-09 `app/api/webhook/route.ts` is the #1 hotspot

Info

BFTB 14

Confirmed

Hotspot

Hotspots · churn-complexity-hotspot · `app/api/webhook/route.ts` · **Open** · V3-E2-S3

Evidence vitals: high complexity, 28 changes in 90 days, co-changes with 14 files.

Impact Central, churning, and complex — a future-bug magnet with wide blast radius.

Fix Decompose into handlers. High value but slow and risky — schedule as a planned refactor, not a quick win.

F-10 No dead code worth pruning

Info

BFTB 20

Confirmed

Dead code · dead-code-clean · `repo-wide` · **Open** · V1-E5-S5

Evidence knip found 3 unused exports, all framework entry points — nothing to remove.

Impact Clean. Noted for transparency; not every module produces a finding.

Fix No action.

Limitations & liability

Engagement terms as supplied for this engagement — no approver recorded.

Scope reviewed

RLS policies, service-role code paths, API routes, auth/permission layer, the booking and billing modules, and the test suite. Reviewed at sample @ demo on 2026-07-30.

What this engagement did not assess

1 of 10 module row(s) were NOT fully assessed on this engagement, and nothing in this report warrants their subject matter: M10 — partial: schema classification ran; live PII-grant check pending client DB sign-off.

SAMPLE REPORT — DEMONSTRATION DATA ONLY.

This document uses a fictional application ("Larkspur") and synthetic findings to illustrate the format and depth of a Harvey audit report. It does not describe a real audit engagement, is not a professional-services deliverable, and creates no engagement relationship,

warranty, or liability of any kind between Harvey and the reader.

A real engagement's report carries counsel-approved engagement terms in this section instead of this notice. To see what Harvey finds in your own codebase, request the free scan at harvey-qa.com.